Code Embeddings for JavaScript Bundle Composition Analysis

Rabie Alawad
Supervisor: Dr. Ben Swierzy
University of Bonn

1 Motivation

Modern websites ship hundreds of third-party npm packages, and a vulnerability in any one is inherited by every site that uses it. Bundlers merge and rewrite these packages into minified files with no dependency manifest, so what a deployed site contains is not directly observable. Because advisories are scoped to exact version ranges, patch-level precision decides whether a site is affected, yet only 5 to 20 percent of production websites update their dependencies within 16 weeks.

Aletheia recovers versions via rolling AST hashes, reaching 87 percent patch-level detection in the lab and 82 percent on real-world bundles, with remaining errors concentrated in small changesets that exact hash overlap cannot resolve. Code embeddings, used in binary SCA tools such as BinaryAI, offer a continuous, learned distance that may tolerate such changes better, and may extend detection to bundles without source maps (RQ2, RQ3).

2 Research Questions

- **RQ1:** Which code representations yield embeddings that remain stable under the transformations bundlers apply, such as minification, tree shaking, and inlining?
- **RQ2:** Under what conditions, if any, do embedding-based approaches improve version detection over existing non-learned approaches such as hashing?
- **RQ3:** Can embeddings identify packages in bundles without source maps, currently an open problem?
- **RQ4:** What trade-offs exist between detection performance and computational cost across different embedding models and mapping techniques?

3 Planned Scientific Work

WP1 Foundations (weeks 1–3): Survey embedding-based matching in binary SCA and current code embedding models; examine the existing implementations and published results of Aletheia, Debun, and other prior approaches as a reference point; explore whether retrieval-augmented generation (RAG) could be applicable to this task; and set up an evaluation harness that supports comparison between embedding-based and other approaches.(RQ2).

WP2 Model & feature selection (weeks 4–5): Test and decide which of two conditions the research is going to move forward with, based on preliminary testing on a pilot set, namely (1) choice of embedding model (e.g,UniXcoder, CodeSage, CodeXEmbed) and (2) choice of code representation fed into the model to generate the embedding (e.g., raw

source tokens, AST-derived features, or other structural representations), scored by how well the resulting embeddings separate adjacent package versions (RQ1).

WP3 Representation fine-tuning (weeks 6–7): As a further condition, test and decide on fine-tuning approaches to improve the selected model's code embeddings, based on preliminary testing on a pilot set across different optimization techniques for separating embeddings, namely contrastive fine-tuning , triplet-loss / metric-learning objectives, supervised contrastive learning , and margin-based losses (RQ1).

WP4 Prototype (weeks 8–11): Assemble the end-to-end pipeline and profile its indexing time, index size, and query latency early, to surface cost issues before the large evaluations (RQ4).

WP5 Lab evaluation (weeks 12–15): Evaluate on bundles with controlled ground truth, targeting the hardest cases for version detection, including minimal changesets, tree shaking, micro-packages, and heavy minification (RQ1, RQ2).

WP6 Real-world evaluation (weeks 16–20): Evaluate on production bundles, drawing on available ground truth where it exists, then test RQ3 by withholding source maps and close with a manual error analysis of the remaining failures (RQ2, RQ3).

WP7 Refinement (weeks 21–24): Tune hybrid weighting and thresholds from the error analysis, reduce index cost, re-run affected measurements (RQ2, RQ4).

WP7 Refinement (weeks 25–26): Contingency buffer for unexpected delays or setbacks.

WP8 Writing (weeks 1–26): Continuous, with related work in WP1, method in WP2–3 and results as produced; final weeks for discussion, limitations and the reproducible artifact.

4 Timeline

6 months, 26 weeks, starting 15 September; each column covers two weeks. Calendar dates are given in the chart below.

